WELT 1 · DOPPELSTUNDE 11 · LEVEL 1.4

W1·2 – Die Heldenschmiede: Vererbung

Datei: `AQ_W1_L1_Heldenschmiede_Geruest.txt` (weiter wie bisher) · IDE-Level 1.4: Basis · Erweiterung · Experte · Im Lab: **Level 1.4**
Niveau (KC 2017): nur Leistungskurs (eA) – im Grundkurs nicht abiturrelevant

Euer Inventar füllt sich – aber eine Fackel ist eben keine Waffe. Schwerter machen Schaden, Tränke heilen, Schlüssel öffnen Türen. Trotzdem haben alle einen Namen und einen Wert in Gold. Statt alles dreimal zu programmieren, nutzt euer Studio **Vererbung**: Waffe, Trank und Schlüssel **sind** Gegenstände – mit ein paar Extras.

Infoteil 1: Vererbung in Java

- Eine **Unterklasse** erbt alle Attribute und Methoden ihrer **Oberklasse** und ergänzt eigene. Es gilt die **„ist ein"-Beziehung**: Eine Waffe ist ein Gegenstand.
- In Java: `public class Waffe extends Gegenstand { … }`
- **Konstruktor:** Die privaten Attribute der Oberklasse (`name`, `wert`) darf die Unterklasse nicht direkt setzen. Darum ruft ihr Konstruktor als **erste Anweisung** `super(…)` auf – den Konstruktor der Oberklasse.
- **Überschreiben:** Die Unterklasse definiert eine Methode mit **gleicher Signatur** neu. Mit `super.methode()` ruft sie dabei die Version der Oberklasse auf.
- **Klassendiagramm (EH § 6):** Pfeil mit **hohler Dreiecksspitze** von der Unterklasse zur Oberklasse. In der Unterklasse stehen nur **neue und überschriebene** Elemente.

Beispiel (gehört nicht zum Projekt-Gerüst): Ein Boss ist ein Monster mit Titel, das nur halben Schaden nimmt.

```
public class Boss extends Monster {
    private String titel;

    public Boss(String name, int leben,
                int staerke, String titel) {
        super(name, leben, staerke);
        this.titel = titel;
    }

    public String getTitel() {
        return titel;
    }

    // ueberschreibt nimmSchaden aus Monster
    public void nimmSchaden(int schaden) {
        super.nimmSchaden(schaden / 2);
    }
}
```

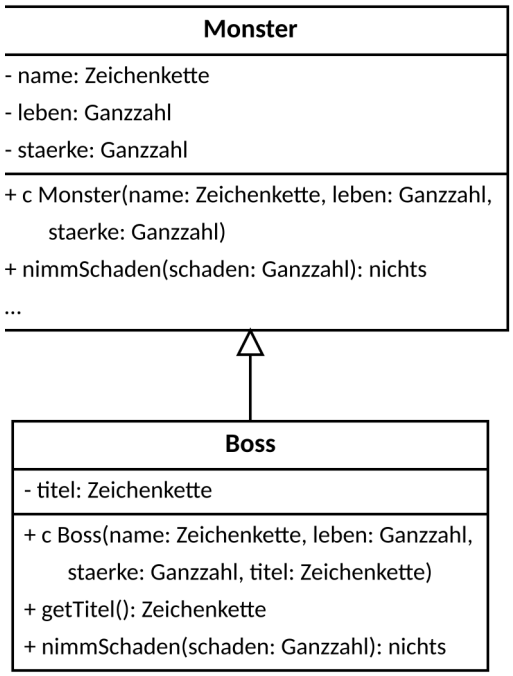


Abbildung 1: Boss erbt von Monster

`super(name, leben, staerke)` setzt die geerbten Attribute. `super.nimmSchaden(schaden / 2)` nutzt die geprüfte Version aus Monster („nie unter 0") – nur mit halbiertem Schaden (ganzzahlige Division).

Infoteil 2: Welche Methode läuft? – Typ der Variable und Typ des Objekts

```
Gegenstand g = new Waffe("Schwert", 50, 12);
println(g.getName());          // geerbt aus Gegenstand: Schwert
println(g.beschreibung());     // ???
// println(g.getSchaden());    // Uebersetzungsfehler!
```

- Der **Typ der Variable** (hier Gegenstand) entscheidet, **welche Methoden aufgerufen werden dürfen**. `getSchaden()` gibt es in Gegenstand nicht – der Compiler lehnt ab.
- Der **Typ des Objekts** (hier Waffe) entscheidet, **welche Version läuft**. Hat die Unterklasse die Methode überschrieben, läuft ihre Version. Das heißt **dynamisches Binden** (Polymorphie).
- Darum liefert `g.beschreibung()` den Text der Waffe: "Waffe: Schwert (50 Gold), Schaden 12".

Zuweisungskompatibilität

Anweisung		Warum?
`Gegenstand g = new Waffe("Axt", 30, 9);`	✓	Jede Waffe ist ein Gegenstand.
`Waffe w = new Gegenstand("Fackel", 5);`	✗	Nicht jeder Gegenstand ist eine Waffe.
`Waffe w2 = g;`	✗	Auch wenn g auf eine Waffe verweist: Der Compiler kennt nur den Typ Gegenstand.
`held.aufnehmen(new Trank("Heiltrank", 20, 30));`	✓	aufnehmen erwartet einen Gegenstand – ein Trank ist einer.

Aufgabe 1: Die Waffenkammer öffnet – Level 1.4

IDE ★ – ★★★

Abbildung 2 zeigt die drei neuen Klassen. Du schreibst sie **komplett selbst** – es gibt im Gerüst nur die TODO-Zeilen.

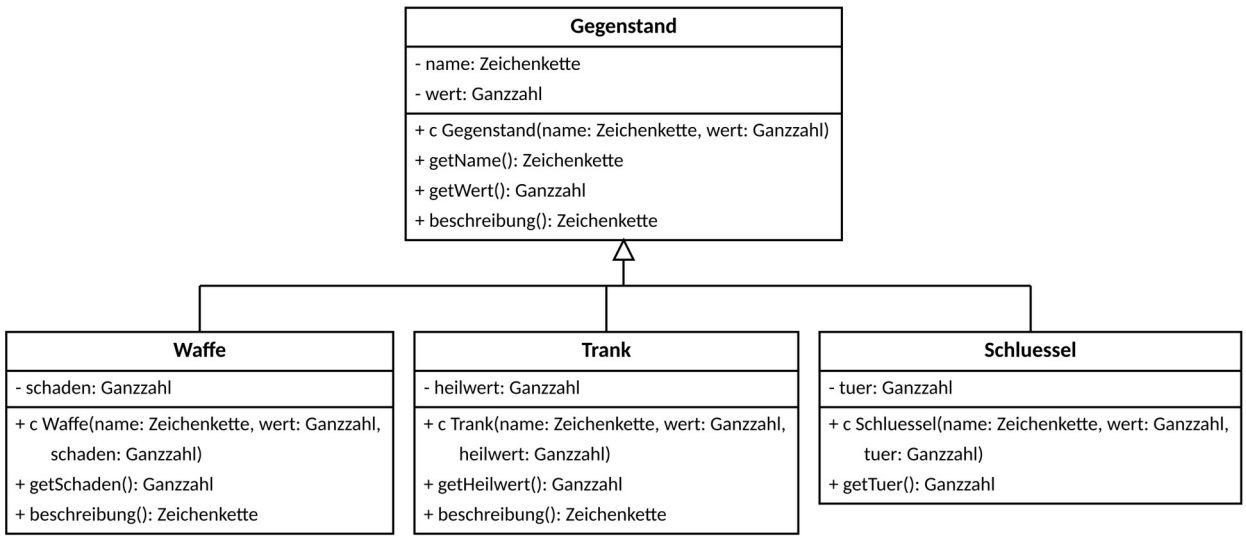


Abbildung 2: Gegenstand und seine Unterklassen

a) Schreibe die Klassen `Waffe`, `Trank` und `Schluessel` nach Abbildung 2. Jeder Konstruktor ruft zuerst `super(name, wert)` auf. Waffe und Trank überschreiben `beschreibung()`, Schluessel nicht:

Objekt	beschreibung() liefert
`new Waffe("Schwert", 50, 12)`	Waffe: Schwert (50 Gold), Schaden 12
`new Trank("Heiltrank", 20, 30)`	Trank: Heiltrank (20 Gold), heilt 30
`new Schluessel("Rostiger Schluessel", 2, 7)`	Rostiger Schluessel (2 Gold)

b) Lege in `Held` das Attribut `waffe` (Typ Waffe) an. Setze es in **beiden** Konstruktoren auf null.

c) **Level 1.4 freischalten:** Wenn deine drei Klassen kompilieren, löschst du in den **drei** markierten Blöcken jeweils genau zwei Zeilen: die Zeile `/* LEVEL-1.4-START` und die Zeile `LEVEL-1.4-ENDE */`. Die Blöcke stehen in der Klasse Held, bei der Testklasse `L14VererbungTest` und im Hauptprogramm. Den Inhalt dazwischen lässt du stehen!

d) Implementiere in Held `getWaffe()`, `ausruesten(Waffe w)`, `trinken(Trank t)` (heilt um den Heilwert) und `inventarAusgeben()` (für jeden Gegenstand eine Zeile mit seiner `beschreibung()`). Erweitere `angriffswert()`: Stärke plus Schaden der Waffe – ohne Waffe nur die Stärke.

e) Starte `L14VererbungTest` (Stufen `basis_` / `erweiterung_` / `experte_` wie gewohnt) und das Hauptprogramm.

Achtung Abi:

- `super(…)` muss die **erste** Anweisung im Konstruktor sein.
- `angriffswert()` braucht eine **null-Abfrage**: Ohne Waffe gibt es keinen Schaden, den man abfragen kann.

Denkanstoß: name und wert sind in Gegenstand privat. Wie kommt die Waffe in `beschreibung()` trotzdem an den Text „Schwert (50 Gold)“?

Aufgabe 2: Der Stammbaum der Gegenstände

PAPIER ★

Abbildung 3 zeigt einen Ausschnitt aus dem Klassendiagramm eures Spiels.

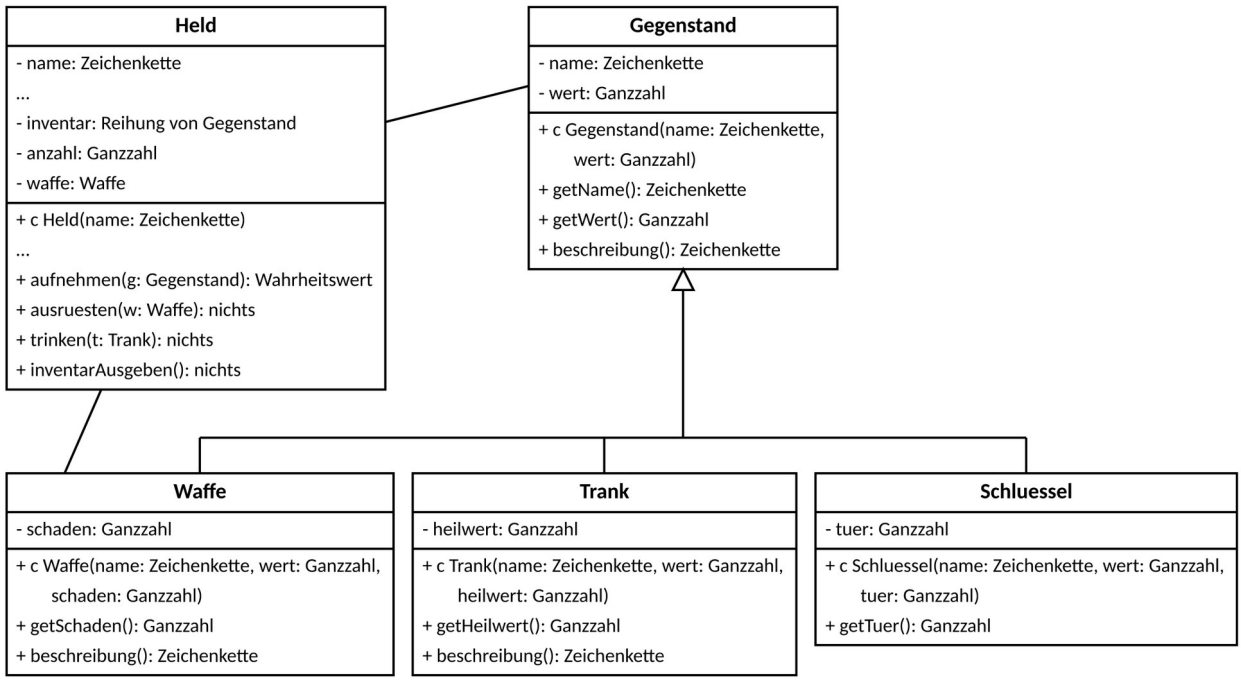


Abbildung 3: Klassendiagramm (Ausschnitt)

a) Nenne alle Attribute und Methoden, die ein Objekt der Klasse Trank besitzt – eigene und geerbte. **[2 BE]**

b) Gib an, welche Methoden der Klasse Gegenstand in Waffe überschrieben und welche nur geerbt werden. **[2 BE]**

c) Die Klasse Held ist durch zwei Linien mit anderen Klassen verbunden. Erläutere, wofür jede Linie steht. **[2 BE]**

d) Entscheide, ob die Anweisung übersetzt wird, und begründe. Es gilt: `Held held = new Held("Aria");` **[3 BE]**

Anweisung	übersetzt? (ja/nein)	Begründung
`Gegenstand g1 = new Trank("Heiltrank", 20, 30);`		
`Trank t = new Gegenstand("Fackel", 5);`		
`Gegenstand g2 = new Schluessel("Rostiger Schluessel", 2, 7);` `int nr = g2.getTuer();`		
`Waffe w = new Waffe("Axt", 30, 9);` `Gegenstand g3 = w;`		
`held.ausruesten(new Trank("Gift", 5, 0));`		

Aufgabe 3: Was zeigt das Inventar?

PAPIER ★★

Gehe davon aus, dass alle Methoden korrekt implementiert sind und `inventarAusgeben()` für jeden Gegenstand genau eine Zeile mit seiner `beschreibung()` ausgibt.

```
1  Held aria = new Held("Aria");
2  Waffe axt = new Waffe("Axt", 30, 9);
3  aria.aufnehmen(new Schluessel("Rostiger Schluessel", 2, 7));
4  aria.aufnehmen(axt);
5  aria.aufnehmen(new Trank("Heiltrank", 20, 30));
6  aria.aufnehmen(new Gegenstand("Fackel", 5));
7  aria.ausruesten(axt);
8  aria.nimmSchaden(45);
9  aria.trinken(new Trank("Elixier", 60, 80));
10 println(aria.getLeben() + " / " + aria.angriffswert());
11 aria.inventarAusgeben();
```

a) Gib die vollständige Ausgabe des Programms an. **[4 BE]**

b) Erläutere am Beispiel der Axt, warum `inventarAusgeben()` den Text der Waffe ausgibt, obwohl `inventar` eine Reihung von Gegenstand ist. **[2 BE]**

c) Gib an, was `aria.gesamtWert()` und `aria.wertvollsterGegenstand().getName()` nach Zeile 11 liefern. Begründe, warum der Elixier-Trank nicht mitzählt. **[2 BE]**

Aufgabe 4: Bug-Hunt in der Rüstkammer

PAPIER ★★

Ein Mitglied eures Studios hat eine Klasse für Rüstungen geschrieben. Darin stecken **drei Fehler**.

```
1  public class Ruestung extends Gegenstand {
2      private int schutz;
3
4      public Ruestung(String name, int wert, int schutz) {
5          this.schutz = schutz;
6      }
7
8      public int getSchutz() {
9          return schutz;
10     }
11
12     public String beschreibung() {
13         return "Ruestung: " + name + " (" + wert
14                 + " Gold), Schutz " + schutz;
15     }
16
17     public boolean istSchild() {
18         return getName() == "Schild";
19     }
20 }
```

a) Gib für jeden Fehler die Zeile an und beschreibe ihn. Entscheide, ob es ein Übersetzungsfehler oder ein Fehler im Verhalten ist. **[3 BE]**

Zeile	Fehler	Übersetzungsfehler / Verhalten	Korrektur

b) Erläutere, warum der Fehler in `istSchild()` in der Online-IDE nicht auffällt, im Abitur aber als Fehler gewertet wird. **[2 BE]**

Aufgabe 5: Abi-Format: Klassendiagramm erweitern

PAPIER ★★★

Euer Studio plant zwei Erweiterungen:

- **Händler:** Ein Händler hat einen Namen und ein Warenlager mit höchstens zehn Gegenständen. Er wird mit seinem Namen erzeugt.
- **Magische Waffen:** Eine magische Waffe ist eine Waffe mit einem Element (z. B. "Feuer") und einem Bonusschaden. Ihr Schaden ist der normale Schaden plus Bonus.

a) Ergänze das Klassendiagramm aus Abbildung 3 um die Klassen Haendler und MagischeWaffe einschließlich aller Beziehungen. Zeichne die vorhandenen Klassen nur als Kasten mit Namen. **[5 BE]**

b) Implementiere den Konstruktor der Klasse MagischeWaffe. **[2 BE]**

c) Der Held soll mit einer magischen Waffe automatisch den Bonusschaden austeilen – ohne dass die Klasse Held geändert wird. Erläutere, welche Methode MagischeWaffe dafür überschreiben muss und warum das funktioniert. **[3 BE]**

🎯 Im Lab üben: Level 1.4 „Die Waffenkammer – Vererbung und Polymorphie“ – Punkte gibt es dort automatisch

Das Lab-Level 1.4 im **Algo Quest Lab** (algoquest.frankjuchim.de) passt zu Infoteil 1–2 und Aufgaben 1–5 dieses Blattes. Die Aufgaben werden automatisch geprüft, Punkte gibt es beim ersten Lösen. Das IDE-Level 1.4 programmierst du weiterhin in der Online-IDE.

Danach wartet im Lab der **Online-Boss „Der Reitstall der Heldenschmiede“** (25 Minuten) – die beste Vorbereitung auf den Papier-Boss.

Für Schnelle

- **Der Boss erwacht:** Übernimm die Klasse `Boss` aus Infoteil 1 in dein Projekt. Schreibe eine eigene Testklasse `BossTest` mit mindestens drei Tests (z. B. 30 Schaden → 15 weniger Leben; 31 Schaden → ganzzahlig 15; ein Boss ist nie unter 0). Side-Quest ohne Punkte.
- **Stilles Überschreiben:** Überschreibe in Schluessel die Methode `beschreibung()`, sodass die Türnummer erscheint. Welcher Test aus `L14VererbungTest` wird rot – und warum? Mach die Änderung danach wieder rückgängig.
- **Denkanstoß:** Wäre `Monster extends Held` eine gute Idee, weil beide name, leben und staerke haben? Begründe mit der „ist ein“-Beziehung.